THE LAUNCHPAD

Smart Notes App

An Automated CI/CD Pipeline That Ships Code While You Sleep

Git / GitHub **Jenkins** **Docker** **CI/CD Pipeline**

1. Project Overview

The Launchpad Protocol is a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline built around a practical, real-world web application — a Smart Notes App that lets users add, edit, and delete personal notes. While the application itself is deliberately simple and relatable, the true engineering showcase lies beneath: every time a developer pushes a single line of code to GitHub, an automated assembly line instantly wakes up, builds the application inside a Docker container, runs safety checks, and deploys the latest version — all without a single manual command.

The pipeline's philosophy is simple: **code should ship itself**. Developers write features; the pipeline handles everything else.

> ***Viva Answer:*** "Whenever I push code to GitHub, Jenkins automatically detects the change, builds the application using a Dockerfile, packages it into a versioned Docker image, and deploys it as a running container — the entire process requires zero manual intervention."

2. Component Roles

The pipeline is assembled from four industry-standard tools, each owning a specific lane in the delivery process:

Tool	Role	Function in the Project
Git / GitHub	The Source of Truth	Stores all source code, the Dockerfile, and the Jenkins automation rules (Jenkinsfile). A git push to GitHub fires the starting pistol for the entire pipeline.

Tool	Role	Function in the Project
Jenkins	The Assembly Manager	Listens for GitHub webhooks, pulls the latest code, triggers Docker builds, runs automated tests, and orchestrates deployment — acting as the brain of the operation.
Docker	The Container Factory	Packages the Flask Notes App and all its dependencies into a portable image. Eliminates the classic "works on my machine" problem by ensuring identical behaviour in every environment.
Flask (Python)	The Application	A lightweight web app providing the Add / Edit / Delete notes interface. Serves as the real-world workload that the entire pipeline builds, tests, and deploys.

3. Deployment Workflow

Every code change travels through a two-phase automated pipeline, moving from a developer's laptop to a live running container:

Phase A — Continuous Integration: Build & Validate

1. **Code Push:** The developer pushes updated code (a new note feature, a bug fix, a UI change) to the GitHub repository.
2. **Webhook Trigger:** GitHub sends an instant webhook notification to Jenkins, signalling that new code is ready.
3. **Jenkins CI Job:** Jenkins clones the latest repository, reads the Jenkinsfile, and starts the automated build factory.
4. **Docker Image Build:** Jenkins executes the Dockerfile — installing Python, copying app files, and packaging everything into a clean, versioned image (e.g. notes-app:v2.1).
5. **Registry Push:** The verified image is pushed to Docker Hub, creating a permanent, retrievable snapshot of that exact version.

Phase B — Continuous Deployment: Containerise & Ship

1. **Pull Latest Image:** Jenkins pulls the freshly built image from Docker Hub onto the target server.
2. **Stop Old Container:** The currently running container is gracefully stopped, freeing the port.
3. **Launch New Container:** Docker spins up a brand-new container from the latest image. The updated Notes App is now live and serving users.
4. **Health Confirmation:** Jenkins hits the /health endpoint to confirm the new container is responding correctly before marking the build as SUCCESS.

4. The Smart Notes Application

The pipeline is demonstrated using a Python Flask web application that provides a clean, browser-based notes management interface. Each endpoint plays a dedicated role in both the user experience and the deployment health-checks:

Endpoint	HTTP Method	Purpose
/	GET	Main user interface — displays all saved notes in the browser.
/add	POST	Accepts a form submission and appends a new note to the list.
/edit/<id>	GET / POST	Loads an existing note for editing and saves the updated content.
/delete/<id>	POST	Removes a specific note by its ID from the collection.
/health	GET	Liveness probe — returns HTTP 200 OK so Jenkins can confirm the container is alive post-deployment.

5. Why This Project Stands Out

Real-World Relevance	Every modern software company uses CI/CD. This project demonstrates the exact workflow that ships code at companies like Netflix, Google, and Amazon — at a scale that can be run on a laptop.
Simple App, Deep Engineering	The Notes App is intentionally minimal so evaluators can focus entirely on the pipeline architecture — the real engineering achievement — without app complexity as a distraction.
Fully Automated, Zero Touch	From git push to live deployment: no manual SSH, no manual docker run commands. The system is self-operating, which is the gold standard of modern DevOps.
Strong Viva Story	The workflow has a clear, memorable narrative arc: code change → GitHub → Jenkins detects → Docker builds → container goes live. Easy to explain, impressive to demonstrate.

6. Core Objective

The Launchpad Protocol transforms software delivery from a manual, error-prone ritual into a fully automated, repeatable machine. By combining version-controlled source code, containerised application packaging, and event-driven orchestration, the system guarantees that every commit either reaches users as a running container — or fails fast with a clear error log, never silently. The result: developers ship with confidence, deployment is no longer a fear event, and the path from idea to live feature is measured in minutes, not hours.